Peer-to-Peer Networks

0xSaad

October 25, 2026

1 Client/Server and Peer-to-Peer

1.1 Client/Server Architecture

A well-known, powerful, reliable server is a data source. Clients request data from the server.

Examples: WWW (HTTP), FTP, and Web services.

1.1.1 Client/Server Limitations

- Scalability is hard to achieve.
- Presents a single point of failure.
- Requires administration.

P2P systems try to address these limitations.

1.2 P2P Computing

P2P computing is the sharing of computer resources and services by direct exchange between systems.

- Resources include information, processing cycles, cache storage, and disk storage for files.
- P2P takes advantage of existing computing power, storage, and networking connectivity.

1.3 P2P Architecture

- All nodes are both clients and servers: they provide and consume data.
- Any node can initiate a connection.
- No centralized data source.

1.3.1 P2P Network Characteristics

- Clients are also servers and routers; nodes contribute content, storage, memory, and CPU.

- Nodes are autonomous (no administrative authority).
- The network is dynamic: nodes frequently enter and leave.
- Nodes collaborate directly, not through well-known servers.
- Nodes have widely varying capabilities.

1.3.2 P2P Benefits

- Efficient use of resources: unused bandwidth, storage, and processing power at the edge of the network.
- Scalability: consumers also donate resources; aggregate resources grow with utilization.
- Reliability: replicas, geographic distribution, and no single point of failure.
- Ease of administration: self-organizing nodes, fault tolerance, replication, and load balancing.

1.3.3 P2P Applications

- File sharing: Napster, Gnutella, Kazaa.
- Multiplayer games: Unreal Tournament, DOOM.
- Collaborative applications: ICQ, shared whiteboard.
- Distributed computation: SETI@home.
- Ad-hoc networks.

Large-scale file sharing may raise issues of copyright infringement.

2 P2P Search Architectures

2.1 Approaches

- Centralized: Napster.
- Flooding or Decentralized: Gnutella.
- Hybrid (Centralized and Decentralized) with Super-Peers: KaZaA.
- Document Routing (Structured): Chord and CAN.

2.2 Napster (Centralized)

A system for sharing music files using centralized searching and direct peer-to-peer file transfer.

1. Users upload their file lists to the Napster server.
2. Users send keyword queries to the server (artist, song, album, bitrate, etc.).
3. The server replies with IP addresses of users with matching files.
4. Users connect directly to another peer to download the file.

2.2.1 Napster Properties

- A central server can ensure correct search results.
- The server is a bottleneck for scalability.
- Single point of failure.
- Susceptible to denial of service, malicious users, lawsuits, and legislation.
- Search is centralized; file transfer is direct (peer-to-peer).

2.3 Gnutella (Flooding)

A decentralized P2P system that shares any type of files, not only music.

1. A peer asks its neighbors for files of interest.
2. Neighbors forward the query to their neighbors, and so on.
3. A TTL (Time To Live) field stops messages after a number of hops.
4. Peers with matching files reply to the requester.

2.3.1 Gnutella Properties

- Decentralized, with no single point of failure.
- Not as susceptible to denial of service.
- Cannot ensure correct results.
- Flooding makes search distributed but still not scalable.

2.4 KaZaA (FastTrack Network)

A hybrid of centralized Napster and decentralized Gnutella.

- Super-Peers act as local search hubs, like Napster servers for small portions of the network.
- Super-Peers are automatically selected by capacity (storage, bandwidth) and availability (connection time).
- Users upload file lists to a Super-Peer.
- Super-Peers periodically exchange file lists.
- Users send file queries to a Super-Peer.

2.5 Free Riding

File-sharing networks rely on users sharing data.

Two types of free riding:

- Downloading but not sharing any data.
- Not sharing any interesting data.

On Gnutella, the study cited in the slides reports:

- 15% of users contribute 94% of content.
- 63% of users never responded to a query.

2.6 Anonymity and Freenet

Napster, Gnutella, and Kazaa do not provide anonymity: users know who they download from, and others know who sent a query.

Freenet is designed to provide anonymity.

- Data flows in the reverse path of the query.
- It is impossible to know if a user is initiating or forwarding a query, or consuming or forwarding data, in the model described in the slides.
- "Smart" queries route requests toward the correct peer by incremental discovery.

2.6.1 Freenet Routing Tables

- `id`: File identifier (e.g., hash of file).
- `next_hop`: Another node that stores the file ID.
- `file`: File identified by the ID, stored at the local node.

Forwarding a query for a file ID:

1. If the file is stored locally, stop and forward data back to the upstream requester.
2. Otherwise, find the closest ID in the routing table and forward to its `next_hop`.
3. If data is not found, report failure; the requester tries the next closest match.

3 Structured P2P and Distributed Hash Tables

3.1 Structured P2P

Second-generation P2P overlay networks based on a Distributed Hash Table (DHT) interface.

- Self-organizing.
- Load balanced.
- Fault-tolerant.
- Scalable guarantees on the number of hops to answer a query, unlike unstructured P2P systems.

3.2 Distributed Hash Tables (DHT)

A distributed version of a hash table data structure that stores `(key, value)` pairs.

- The key is like a filename; the value can be file contents.
- Goal: Efficiently insert, lookup, and delete `(key, value)` pairs.
- Each peer stores a subset of the key-value pairs.

- Core operation: Find the node responsible for a key.
- Map a key to a node and efficiently route requests to that node.

3.2.1 Directed Searches

- Assign particular nodes to hold particular content, or know where it is.
- A request goes toward the node responsible for the content.
- Challenges: Evenly distributing responsibilities and adapting to nodes joining, leaving, or failing.
- Give responsibilities to joining nodes; redistribute responsibilities from leaving nodes.

3.2.2 Hash Tables and DHTs

- A hash table associates data with keys and hashes a key to find a bucket.
- Each bucket is expected to hold items/buckets items.
- In a DHT, nodes are the hash buckets: a key is hashed to find its responsible peer.
- Data and load are balanced across nodes.

3.2.3 DHT Applications

- File sharing and archival storage.
- Databases.
- Naming and service discovery.
- Chat service.
- Rendezvous-based communication.
- Publish/Subscribe.

3.2.4 DHT Desirable Properties

- Keys are mapped evenly to nodes (e.g., 10 keys and 5 peers: 2 keys per peer).
- Each node maintains information about only a few other nodes.
- Messages are routed efficiently.
- Node arrivals and departures affect only a few nodes.

3.2.5 DHT Routing Protocols

DHT is a generic interface with several implementations:

- Chord (MIT).
- Pastry (Microsoft Research UK, Rice University).
- Tapestry and CAN (UC Berkeley).
- SkipNet, Kademlia, Viceroy, P-Grid, and Freenet.

These systems are called P2P routing substrates or P2P overlay networks.

4 Chord

4.1 Chord API

- Node ID: Unique m-bit identifier (hash of IP address or another unique ID).
- Key: m-bit identifier (hash of a sequence of bytes).
- Value: Sequence of bytes.

Operations:

- `insert(key, value)`: Store key/value at r nodes.
- `lookup(key)`.
- `update(key, newval)`.
- `join(n)`.
- `leave()`.

4.2 Chord Properties

In a system with N nodes and K keys, with high probability, the slides give:

- Key responsibility on the order of K/N per node.
- Each node maintains information about $O(\log N)$ other nodes.
- Lookups are resolved in $O(\log N)$ hops.
- No delivery guarantees.
- No consistency among replicas.

4.3 Chord Identifier Ring

Chord uses a circular m-bit ID space for both keys and nodes:

$$0, 1, \ldots, 2^m - 1.$$

- Node ID: SHA-1 of an IP address (as shown in the slides).
- Key ID: SHA-1 of a key.
- A key maps to the first node whose ID is equal to or follows the key ID on the ring.
- Each node is responsible for $O(K/N)$ keys; $O(K/N)$ keys move when a node joins or leaves.

Example: $m = 6$, so $2^6 = 64$ possible IDs (0–63).

With nodes $\{1, 8, 14, 21, 32, 38, 42, 48, 51, 56\}$, the mappings illustrated in the slides include:

$$K10 \to N14, \quad K24 \to N32, \quad K30 \to N32, \quad K38 \to N38, \quad K54 \to N56.$$

4.4 Chord State and Lookup

4.4.1 Basic Chord

- Each node knows its Successor and Predecessor (for ring management).
- Lookup requests are forwarded around the ring through Successor pointers.
- Requires $O(N)$ hops.

4.4.2 Finger Table

- Each node knows m other positions on the ring.
- Finger i of node n points to the Successor of $n + 2^i$.
- Lookup follows the closest preceding fingers and then the Successor.
- Requires $O(\log N)$ hops.

$$\text{finger}_i(n) = \text{successor}\big((n + 2^i) \bmod 2^m\big), \qquad i = 0, \ldots, m-1.$$

Finger table for $N8$ when $m = 6$, as in the slides:

i	$N8 + 2^i$	Successor
0	9	N14
1	10	N14
2	12	N14
3	16	N21
4	24	N32
5	40	N42

4.5 Chord Join and Routing

In the identifier space $[0, 7]$, $m = 3$.

- Node $n1$ joins first: every Successor-table entry points to $n1$.
- Node $n2$ joins: the Successor tables change.
- Nodes $n0$ and $n6$ join: active nodes are $\{n0, n1, n2, n6\}$.
- Items $f1$ and $f7$: $f1$ maps to $n1$, and $f7$ maps to $n0$.

Example Successor tables for the four active nodes (each row lists entries for $i = 0, 1, 2$):

Node	Successor-table entries
$n0$	$n1,\ n2,\ n6$
$n1$	$n2,\ n6,\ n6$
$n2$	$n6,\ n6,\ n6$
$n6$	$n0,\ n0,\ n2$

Upon receiving a query for an item ID:

1. Check if the item is stored locally.
2. Otherwise, forward to the largest preceding node in the Successor table.
3. Continue until the responsible node is reached.

Example from the slides: Query for item 7 from $n1$ is routed toward $n6$, then $n0$.

4.6 Network Locality

Nodes close on the Chord ring can be far apart in the physical network.

5 Content Addressable Network (CAN)

5.1 Document Routing – CAN

CAN associates each node and item with a unique identifier in a d-dimensional space.

Goals:

- Scale to hundreds of thousands of nodes.
- Handle rapid arrival and failure of nodes.

Properties:

- Routing table size: $O(d)$.
- File location in at most $d\,n^{1/d}$ routing steps, where n is the total number of nodes, as stated in the slides.

5.2 CAN Two-Dimensional Space

- The space is divided between nodes; all nodes together cover the entire space.
- Each node covers a square or rectangular area (ratios 1:2 or 2:1 in the slide example).
- The first node, $n1 : (1, 2)$, covers the entire space.
- When $n2 : (4, 2)$ joins, the space is divided between $n1$ and $n2$.
- Later nodes join: $n3 : (3, 5)$, $n4 : (5, 5)$, and $n5 : (6, 6)$.

5.2.1 Data Placement

Each item is stored by the node that owns its mapped area in the coordinate space.

Item	Coordinates	Owning Node
$f1$	$(2, 3)$	$n1$
$f2$	$(5, 1)$	$n2$
$f3$	$(2, 1)$	$n1$
$f4$	$(7, 5)$	$n5$

5.3 CAN Query Routing

- Each node knows its neighbors in the d-dimensional space.
- Forward a query to the neighbor closest to the query ID.
- Slide example: $n1$ queries item $f4$.
- CAN can route around some failures; some failures require local flooding.

6 P2P Web

Works like an ordinary Web server, with these differences:

- Distributed storage.
- Automatic data replication.
- Load balancing of users.
- When the load is too high, add another server and store another copy.